41 SUPABASE POSTGRES + NESTJS STUDENT CRUD — EVERY STEP, WITH THE THEORY

theory/40 is the general "zero to CRUD" walkthrough (Docker / local / hosted). This file is the REAL build done in this project: a hosted Supabase Postgres database, wired into the existing NestJS app next to Mongo, with a full student CRUD under /student-pg. For every step: WHAT you do, WHY it's needed, and WHAT HAPPENS under the hood.

The full path, in order:

```
Supabase project -> connection string -> .env -> packages
-> TypeOrmModule connection -> @Entity -> DTOs -> forFeature()
-> Repository service -> controller -> AppModule -> run -> test
```

STEP 1 — CREATE THE DATABASE SERVER (SUPABASE)

WHAT

```
supabase.com -> New project -> pick a name, a database password and
a region (this project uses ap-northeast-1 = Tokyo).
```

THEORY

- Supabase is "hosted Postgres": a real PostgreSQL server (v17 here) running in the cloud. Nothing is installed on your machine.
- Creating the project gives you a SERVER and a default DATABASE named `postgres` already inside it — so theory/40 STEP 2 (CREATE DATABASE) is not needed; we just use `postgres`.
- The password you choose here is the password for the `postgres` database user. Save it — Supabase does not show it again (you can only reset it).

STEP 2 — GET THE CONNECTION STRING

WHAT

Project → Connect → copy the "Session pooler" URI:

postgresql://postgres.<project-ref>:[YOUR-PASSWORD]@aws-0-ap-northeast-1.pooler.supabase.com:5432/postgres

THEORY — reading the URI piece by piece:

```
postgresql://                  protocol — tells the driver "this is Postgres"
postgres.<project-ref>         USER. On the pooler the user is
                               "postgres" + "." + your project id, so the
                               pooler knows which project to route to
:[YOUR-PASSWORD]               the DB password from STEP 1
@aws-0-...pooler.supabase.com  HOST — Supabase's connection pooler
                               (Supavisor), not the database machine itself
:5432                          PORT — 5432 = session mode (default Postgres port)
```

```
/postgres                    DATABASE name inside the server

Why a POOLER?
- A Postgres server can only hold a limited number of open
  connections. A pooler sits in front and shares a small set of real
  DB connections between many clients.
- Session mode (port 5432): you keep one real connection for your
  whole session. Works with everything TypeORM does. Use this for a
  long-running NestJS server.
- Transaction mode (port 6543): connection is returned after every
  transaction. Good for serverless; breaks prepared statements.
- The pooler is also IPv4-friendly; the "direct" connection
  (db.<ref>.supabase.co) is IPv6-only on free tier, which many home
  networks can't reach.
```

Password rule: if the password has special characters (@ # / : ? %) they must be URL-encoded inside the URI (@ → %40, # → %23), or the driver will split the URI at the wrong place.

STEP 3 — PUT IT IN .env

WHAT

.env now holds both databases side by side:

```
PORT=3000
MONGODB_URI="mongodb://..."
DATABASE_URL="postgresql://postgres.<ref>:<password>@aws-0-ap-northeast-
1.pooler.supabase.com:5432/postgres"
```

THEORY

- Same pattern as MONGODB_URI (theory/28, theory/29): secrets live in .env, code only knows the variable NAME.
- ConfigModule.forRoot({ isGlobal: true }) reads .env once at boot, so ConfigService can hand DATABASE_URL to TypeORM later.
- .env is listed in .gitignore → the password never gets committed. If a password ever leaks (pasted in chat, screenshot, commit), reset it in Supabase → Project Settings → Database and update .env.

Quick connection test BEFORE writing any Nest code (proves URI + password + network are right, so later errors must be in code):

```
node -e "require('dotenv').config();const {Client}=require('pg');
  const c=new Client({connectionString:process.env.DATABASE_URL,ssl:{rejectUnauthorized:false}});
  c.connect().then(()=>c.query('select version()')).then(r=>{console.log(r.rows[0].version);c.end()})"

-> PostgreSQL 17.x on aarch64-unknown-linux-gnu ...
```

STEP 4 — PACKAGES

WHAT

```
npm install @nestjs/typeorm typeorm pg
```

THEORY — three layers, top to bottom:

```
@nestjs/typeorm   Nest glue: TypeOrmModule.forRootAsync / forFeature,
                  @InjectRepository. Same role as @nestjs/mongoose.
typeorm           the ORM: @Entity classes, Repository, turns method
                  calls (find, save, delete) into SQL. Same role as
                  mongoose.
pg                the raw driver: opens the TCP socket, speaks the
                  Postgres wire protocol, sends SQL text, gets rows
                  back. Same role as the mongodb driver.
```

Your code → Repository (typeorm) → SQL → pg → network → Supabase

STEP 5 — CONNECT IN app.module.ts (TypeOrmModule.forRootAsync)

WHAT (added next to the existing MongooseModule)

```
TypeOrmModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (configService: ConfigService) => ({
    type: 'postgres',
    url: configService.getOrThrow<string>('DATABASE_URL'),
    ssl: { rejectUnauthorized: false },
    autoLoadEntities: true,
    synchronize: true,
  }),
}),
StudentPgModule,
```

THEORY — each option:

```
forRootAsync + inject + useFactory
    "async" form: Nest first builds ConfigService, then calls
    useFactory with it. Needed because the URL comes from .env at
    runtime, not a hard-coded string. Identical pattern to
    MongooseModule.forRootAsync in the same file.
type: 'postgres'
    which driver TypeORM should load (pg).
url
    the whole connection string; TypeORM parses user/pass/host/port/db
    out of it. getOrThrow -> app refuses to boot if DATABASE_URL is
    missing, instead of failing later with a confusing error.
ssl: { rejectUnauthorized: false }
    Supabase only accepts encrypted (SSL/TLS) connections over the
    internet. rejectUnauthorized:false = encrypt, but don't verify the
    certificate chain. Fine for learning; for production, download
    Supabase's CA certificate and pass it as ssl: { ca: ... }.
autoLoadEntities: true
    every entity passed to any TypeOrmModule.forFeature([...]) is
    registered automatically — no list of entities here.
synchronize: true
    on every boot TypeORM compares your @Entity classes with the real
    tables and runs CREATE TABLE / ALTER TABLE to make them match.
    This is how the `students` table appeared without writing SQL.
    DEV ONLY — renaming a property can DROP a column and its data.
    Production uses migrations (theory/40 STEP 12).
```

Two databases in one app:

```
MongooseModule and TypeOrmModule are completely independent
connections. /student keeps using Mongo, /student-pg uses
Postgres. Nothing about Mongo was removed.
```

STEP 6 — THE ENTITY = THE TABLE (src/student-pg/student.entity.ts)

WHAT

```ts
@Entity('students')
export class Student {
  @PrimaryGeneratedColumn() id!: number;
  @Column()                 name!: string;
  @Column()                 age!: number;
  @Column({ unique: true }) email!: string;
  @Column({ nullable: true }) course?: string;
  @CreateDateColumn()       createdAt!: Date;
  @UpdateDateColumn()       updatedAt!: Date;
}
```

THEORY — what SQL synchronize generated from this:

```sql
CREATE TABLE "students" (
  "id"        SERIAL PRIMARY KEY,
  "name"      varchar NOT NULL,
  "age"       integer NOT NULL,
  "email"     varchar NOT NULL UNIQUE,
  "course"    varchar NULL,
  "createdAt" timestamp NOT NULL DEFAULT now(),
  "updatedAt" timestamp NOT NULL DEFAULT now()
);
```

Decorator → SQL mapping:

@Entity('students')	the table name
@PrimaryGeneratedColumn()	SERIAL PRIMARY KEY: auto-number 1, 2, 3... Postgres needs an explicit PK; Mongo gave you _id for free.
@Column()	NOT NULL by default; type comes from the TS type (string → varchar, number → integer)
unique: true	a real UNIQUE constraint — Postgres itself rejects a duplicate email, even from SQL typed directly in Supabase
nullable: true	column allows NULL → optional field
@CreateDateColumn	filled once on INSERT
@UpdateDateColumn	refreshed on every UPDATE

> Key difference vs Mongo: this schema is HARD. Every row has exactly these columns, enforced by the database, not just by the app.

STEP 7 — DTOs (validation before the DB is touched)

WHAT

Reused the existing src/student/dto/create-student.dto.ts and update-student.dto.ts — no new DTO files.

THEORY

- DTOs describe the REQUEST BODY; the entity describes the TABLE. They are different jobs, so they are different classes.
- class-validator doesn't care which database is behind it, so the same rules (IsString, IsInt, IsEmail, IsOptional) work for both.
- The global ValidationPipe in main.ts (whitelist + forbidNonWhitelisted) runs BEFORE the controller: bad email → 400 "email must be an email", no SQL is sent extra field → 400, field rejected
- UpdateStudentDto = PartialType(CreateStudentDto): all fields optional for updates, rules still applied to whatever is sent.

Two layers of protection:

DTO (app level)	→ catches bad shape/format early, nice messages
Constraints (DB)	→ NOT NULL / UNIQUE, last line of defence

STEP 8 — MODULE: REGISTER THE ENTITY (student-pg.module.ts)

WHAT

```
@Module({
  imports: [TypeOrmModule.forFeature([Student])],
  providers: [StudentPgService],
  controllers: [StudentPgController],
})
export class StudentPgModule {}
```

THEORY

- forFeature([Student]) creates a Repository<Student> provider inside this module's DI container. Without it, @InjectRepository(Student) fails with "Nest can't resolve dependencies of StudentPgService".
- It also tells autoLoadEntities (STEP 5) that this entity exists, so synchronize builds its table.
- Same role as MongooseModule.forFeature([{ name, schema }]) in student.module.ts.

STEP 9 — SERVICE: CRUD WITH Repository<Student> (student-pg.service.ts)

WHAT + the SQL each call becomes:

```
constructor(@InjectRepository(Student) private studentRepo: Repository<Student>) {}

CREATE
  const student = this.studentRepo.create(dto);  // in memory only, NO SQL
  return this.studentRepo.save(student);
    -> INSERT INTO "students"("name","age","email","course",...)
       VALUES ($1,$2,$3,$4) RETURNING "id","createdAt","updatedAt"
```

```
READ ALL
  return this.studentRepo.find();
    -> SELECT * FROM "students"

READ ONE
  const student = await this.studentRepo.findOneBy({ id });
    -> SELECT * FROM "students" WHERE "id" = $1 LIMIT 1
  if (!student) throw new NotFoundException(...)   // null -> 404

UPDATE
  const student = await this.studentRepo.preload({ id, ...dto });
    -> SELECT the row by id, then MERGE dto's fields on top (in memory)
  if (!student) throw new NotFoundException(...)
  return this.studentRepo.save(student);
    -> UPDATE "students" SET "age"=$1, "updatedAt"=now() WHERE "id"=$2
       (only changed columns are written)

DELETE
  const student = await this.findOne(id);   // 404 if missing + keeps a copy
  await this.studentRepo.delete(id);
    -> DELETE FROM "students" WHERE "id" = $1
  return student;                           // return what was deleted
```

THEORY

- $1, $2 are PARAMETERS: values are sent separately from the SQL text, so user input can never change the query (protection against SQL injection). TypeORM does this for you.
- Every method is async / returns a Promise because each one is a network round-trip to Supabase (theory/32, theory/34).
- create() vs save(): create() only builds the object; save() is the one that talks to the database.
- preload() is why PATCH works with partial bodies: it loads the current row first, so fields you didn't send keep their old values.

STEP 10 — CONTROLLER: ROUTES (student-pg.controller.ts)

WHAT

```
@Controller('student-pg')
  POST   createstudent              -> create(dto)
  GET    getallstudents             -> findAll()
  GET    getstudent/:id             -> findOne(id)
  PUT    updatestudent/:id          -> update(id, dto)
  PATCH  updatestudent/:id          -> update(id, dto)
  DELETE deletestudent/:id          -> remove(id)

@Param('id', ParseIntPipe) id: number
```

THEORY

- Same route names as the Mongo controller, different prefix (/student vs /student-pg), so both can be compared side by side.
- ParseIntPipe: URL params always arrive as STRINGS ("1"). Postgres ids are integers, so the pipe converts "1" → 1. "abc" → automatic 400 "Validation failed (numeric string is expected)" before the service runs. Mongo

didn't need this because its _id is a string.

- PUT and PATCH call the same method here because the DTO is already partial (theory/35 explains the difference in meaning).

STEP 11 — REGISTER THE MODULE IN AppModule

WHAT

```
import { StudentPgModule } from './student-pg/student-pg.module';
imports: [ ..., TypeOrmModule.forRootAsync({...}), StudentPgModule ]
```

THEORY

- A module that isn't imported into the tree starting at AppModule doesn't exist to Nest: its routes return 404.
- Order at boot: ConfigModule loads .env → TypeOrmModule connects to Supabase → synchronize creates/updates tables → StudentPgModule gets its Repository → routes are mapped → app.listen(3000).
- If the DB connection fails (wrong password, no internet), TypeORM retries a few times and then the app fails to start — it never starts "half working".

STEP 12 — RUN AND TEST (results from the real run)

```
  npm run start:dev

  POST /student-pg/createstudent
{"name":"Asha","age":21,"email":"asha.pgtest@example.com","course":"CS"}
    -> {"id":1, ..., "course":"CS", "createdAt":..., "updatedAt":...}
  POST /student-pg/createstudent  {"name":"Ravi","age":22,"email":"ravi.pgtest@example.com"}
    -> {"id":2, ..., "course":null}          // optional column -> NULL
  GET  /student-pg/getallstudents  -> [ row 1, row 2 ]
  GET  /student-pg/getstudent/1    -> row 1
  PATCH /student-pg/updatestudent/1  {"age":23}
    -> age 23, updatedAt changed, createdAt unchanged
  DELETE /student-pg/deletestudent/1 -> returns the deleted row
  GET  /student-pg/getstudent/1    -> 404 "Student 1 not found"
  POST with "email":"nope"          -> 400 "email must be an email"

curl versions (Windows cmd needs \" inside the JSON):
  curl -X POST http://localhost:3000/student-pg/createstudent -H "Content-Type: application/json" -d "
{\"name\":\"Asha\",\"age\":21,\"email\":\"asha@example.com\"}"
  curl http://localhost:3000/student-pg/getallstudents
  curl -X PATCH http://localhost:3000/student-pg/updatestudent/2 -H "Content-Type: application/json" -d
"{\"age\":23}"
  curl -X DELETE http://localhost:3000/student-pg/deletestudent/2
```

See the data in Supabase:

```
Dashboard -> Table Editor -> students        (spreadsheet view)
Dashboard -> SQL Editor -> SELECT * FROM students;
```

STEP 13 — ONE REQUEST, START TO FINISH (PATCH /student-

pg/updatestudent/1)

1. HTTP request arrives at Express (port 3000)
2. LoggeerMiddleware logs it (runs on every route)
3. Router matches @Controller('student-pg') + @Patch('updatestudent/:id')
4. ParseIntPipe: "1" → 1
5. ValidationPipe: body {"age":23} checked against UpdateStudentDto
6. StudentPgController.patch(1, dto) → StudentPgService.update(1, dto)
7. preload → SELECT ... WHERE id = 1 (over SSL, via the Supabase pooler)
8. merge age=23 in memory
9. save → UPDATE students SET age=23, updatedAt=now() WHERE id=1
10. Postgres checks constraints, writes the row, returns it
11. Nest serialises the entity to JSON → 200 response

⚠ COMMON ERRORS AND WHAT THEY MEAN

```
password authentication failed for user "postgres.<ref>"
    wrong password, or [YOUR-PASSWORD] brackets left in the URI
Tenant or user not found
    user part is wrong — on the pooler it must be postgres.<project-ref>
getaddrinfo ENOTFOUND db.<ref>.supabase.co
    using the IPv6-only direct host; switch to the pooler URI
self-signed certificate in certificate chain / no pg_hba.conf entry ... SSL off
    missing ssl option in TypeOrmModule config
duplicate key value violates unique constraint
    email already exists — Postgres UNIQUE doing its job (returns 500
    unless you catch it; could map it to 409 Conflict)
Nest can't resolve dependencies of StudentPgService (?)
    forgot TypeOrmModule.forFeature([Student]) in the module
Cannot GET /student-pg/...
    StudentPgModule not imported in AppModule
Validation failed (numeric string is expected)
    non-number id in the URL — ParseIntPipe rejected it
```

FILES CREATED / CHANGED

.env	+ DATABASE_URL
src/app.module.ts	+ TypeOrmModule.forRootAsync, + StudentPgModule
src/student-pg/student.entity.ts	the table
src/student-pg/student-pg.module.ts	forFeature registration
src/student-pg/student-pg.service.ts	Repository CRUD

src/student-pg/student-pg.controller.ts	routes
(reused) src/student/dto/*.ts	validation

★ RULE OF THUMB

Supabase project → pooler URI → .env → packages → forRootAsync (url + ssl) → @Entity (= table) → DTO (= request body) → forFeature (= repository in DI) → service (repository calls = SQL) → controller (routes + ParseIntPipe) → import in AppModule → run, test, check the Table Editor. Mongo and Postgres live side by side; each step mirrors something you already built for Mongo.

See also: **theory/40-postgres-end-to-end.txt** (generic setup), **theory/36-postgres-setup-and-crud.txt** (Mongo vs Postgres CRUD), **theory/37-sql-cheatsheet.txt** (raw SQL), **theory/35-put-vs-patch.txt**, **theory/28-environment-variables.txt**, **theory/29-nestjs-config-module.txt**, **theory/13-dto.txt**